

WSL2 / Ubuntu OpenClaw Setup Guide

From a clean Windows machine to a stable local AI agent

OldGenAI · @OldGenAI · £12

Chapter 1: Why This Guide?

Page 1: The Promise

If you're reading this, you've probably already spent the last six hours staring at a terminal window, watching your RAM evaporate until your whole system crawls to a halt. You tried to follow a random YouTube tutorial, or maybe you just thought, "How hard can it be?"

It was harder than you thought.

This guide isn't about the theory of LLMs. It's about survival. I'm not here to teach you how transformers work; I'm here to make sure that when you finally get your agent running on Windows, it actually stays running.

Who this is for

It's for the person starting with a clean Windows installation and a desire to run high-quality local models without losing their mind. It's for those who aren't afraid of a terminal but are tired of broken configs and wiped workspaces. If you can follow instructions and don't mind getting deep into the guts of WSL2, you're my target audience.

What you will have at the end

By the time you finish this guide, you won't just have "installed" software. You will have a stable, resilient agent environment. Specifically:

- **A rock-solid WSL2/Ubuntu setup** that doesn't break every time you reboot.

- **OpenClaw properly integrated with LM Studio**, with the networking configured so they actually talk to each other.
- **The "Stability Stack"**: You'll have implemented the specific fixes—from heartbeat management to context-bloat prevention—that stop the loops and RAM blowouts nobody else is documenting.
- **A predictable baseline**: A Gemma 4 26B-A4B setup that hits ~40s cold starts and ~2–3s warm responses.

What this guide does NOT cover

I value your time, so let's be clear about the boundaries:

- **This is not a Mac guide.** (That's coming later).
- **This is not a deep dive into Python or Node.js development.** We are configuring, not coding.
- **This is not about hardware optimization for specific GPUs.** We are focusing on the WSL2/Ubuntu/LM Studio pipeline.
- **I am not here to fix your broken Windows drivers.** If your base OS is a mess, start there before opening this guide.

We build well, or we don't ship. Let's get to work.

Page 2

I'd already done this on a Mac Mini the week before. One liner from the official site, pasted straight into the terminal. Done. So when I sat down with Windows, I had that dangerous thing — just enough confidence.

Pasted the same command. It worked. For about ten seconds. Then: *no Homebrew found*.

Turns out Windows doesn't do Homebrew. That's a macOS thing. So I went via npm instead — except Node.js wasn't installed. Fine. Installed Node. The install kept failing. An hour later I found out running as Administrator fixed it. An hour. For that.

Got OpenClaw running. Got LM Studio installed — which, if you've never touched a local inference engine, is genuinely intimidating. Downloaded a model. Thought I was ready.

Then I realised I'd installed everything directly on Windows. Not sandboxed in WSL2 and Ubuntu. I didn't even know what WSL2 was.

That was the *what the hell am I doing* moment.

In the wreckage of that, I remembered something from a Lex Fridman podcast — the OpenClaw creator saying: *if you have a problem, ask the AI. It's infinitely patient*. So I did. That's when I found Claude. That's when my eyes actually started to open.

It was far from the end of my problems. But it was the end of facing them alone.

 Note

Already running WSL2 and Ubuntu?

Skip to Chapter 3.

Chapter 2: WSL2 + Ubuntu from a clean Windows machine

1. Prerequisites & Enable WSL2

Before starting, ensure your host system meets the minimum requirements. WSL2 requires **Windows 10 build 19041 or higher**, or **Windows 11**.

Open PowerShell as Administrator and run:

```
ws1 --install
```

Restart your computer when prompted. This process enables the Virtual Machine Platform, enables the Windows Subsystem for Linux feature, and installs the default Ubuntu distribution.

 Warning

Always snapshot your workspace or create a fundamental backup before making major configuration changes to your system or WSL environment.

2. Verify Installation & Version

Once you have rebooted, you need to ensure that the subsystem is actually running version 2. Open PowerShell (or Command Prompt) and run:

```
ws1 -l -v
```

Expected Output: You should see a list containing `Ubuntu` with a `VERSION` column showing `2`.

If you see `1` under the version column, you are running in compatibility mode, which will cause significant networking and performance issues for OpenClaw. You can convert it using:

```
ws1 --set-version Ubuntu 2
```

3. Install Ubuntu & User Setup

If Ubuntu did not launch automatically after your restart:

1. Open the **Microsoft Store**.
2. Search for "**Ubuntu**" (the latest LTS version is recommended).
3. Click **Get** or **Install**.
4. Once installed, launch Ubuntu from your Start Menu.
5. A terminal window will open to finish installation. Follow the prompts to create a unique **username** and **password**.

Note: This password is for `sudo` commands within Linux; do not forget it.

4. Basic Shell Survival & Environment Setup

You are now operating inside a Linux environment. To ensure your environment is ready for agentic engineering, you must update your package lists and install baseline utilities.

Run the following block in your Ubuntu terminal:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git build-essential
```

Core Commands to Know

Familiarize yourself with these essential navigation commands:

- `pwd` : **P**rint **W**orking **D**irectory (Where am I?).
- `ls -la` : **L**ist all files in the current directory, including hidden configuration files.
- `cd <path>` : **C**hange **D**irectory to a specific folder.
- `mkdir <name>` : **M**ake a new **d**irectory (folder).

Accessing Windows Files

One of the most powerful features of WSL2 is its ability to reach into your Windows host. Your Windows drives are automatically mounted under the `/mnt/` directory.

- To access your C: drive, use: `cd /mnt/c/`
- For example, if you keep your project notes in Obsidian on Windows, you can navigate there directly from Ubuntu via: `cd /mnt/c/Users/<YourUsername>/Documents/...`

⚠ Warning

The Token Mismatch Trap:

Be extremely vigilant about environment variables. API keys, authentication tokens, or paths set as environment variables on your Windows host (e.g., in Windows System Properties) are

not

automatically available inside WSL2. One environment does not see the other's secrets. You must define these explicitly within your `.bashrc` or `.env` files inside Ubuntu.

5. The Final Verification

Before moving to Chapter 3, run this final check in your Ubuntu terminal to confirm that the subsystem is correctly configured, you are in your Linux home directory, and you have connectivity to your Windows filesystem:

```
# Check if we are in /home/user, verify WSL2 version via kernel check, and test Windows access
pwd && uname -r && ls /mnt/c/ | head -n 5
```

Success Criteria:

1. The output shows your Linux home path (e.g., `/home/yourname`).
2. The `uname -r` command returns a kernel string containing `-microsoft-standard-WSL2` .
3. You see a list of Windows folders (like `Users` , `Program Files` , etc.) from the `/mnt/c/` check.

If these pass, your foundation is solid. You are ready for Chapter 3.

Sidebar: The Windows-native Trap

Do not attempt to run OpenClaw directly on your Windows host. While it may appear to work initially with standard Windows CMD or PowerShell, you will encounter severe networking and permission friction when trying to connect to LM Studio or other WSL2-based services. Always perform your primary agentic engineering inside the Ubuntu/WSL2 environment.

 Tip

If OpenClaw is installed, the gateway service is running, and LM Studio is communicating across your WSL2/Windows boundary, move directly to Chapter 4.

Chapter 3: The Core Stack (Node, OpenClaw, and LM Studio)

This chapter covers the installation of the primary execution environment (WSL2), the agent framework (OpenClaw), and the inference engine (LM Studio), along with the critical networking configuration required to bridge them.

1. Node.js Installation (WSL2)

Do not use the default Ubuntu `apt` repository; it is often outdated. Use `nvm` (Node Version Manager) to ensure compatibility with OpenClaw's requirements.

1. Install `nvm` :

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

2. Reload your shell configuration:

```
source ~/.bashrc
```

3. Install the LTS version (or specific v22+):

```
nvm install 22  
nvm use 22
```

4. Verify installation:

```
node -v
```

⚠ Important

Note that any Node.js installation you have on your Windows host is entirely separate from this WSL2 environment. You must not attempt to point WSL2 to the Windows-native `node.exe`. The two environments are isolated; for OpenClaw to function correctly within the Linux subsystem, its dependencies must reside natively within the Ubuntu file system.

2. OpenClaw Installation (WSL2)

OpenClaw installs as a global npm package and runs as a systemd service that starts automatically every time you open Ubuntu.

Enable systemd in WSL2

WSL2 needs systemd enabled for the auto-start service to work. Check first:

```
cat /etc/wsl.conf
```

If the file doesn't contain `systemd=true`, add it:

```
sudo nano /etc/wsl.conf
```

Add the following (create the file if it doesn't exist):

```
[boot]
systemd=true
```

Save, exit, then restart WSL2 from PowerShell on Windows:

```
wsl --shutdown
```

Reopen Ubuntu before continuing.

Install OpenClaw

```
npm install -g openclaw
```

Set Up the Gateway Service

Once installed, configure OpenClaw to run as a background service. First, get your node path:


```
which node  
npm root -g
```

Note both outputs — you'll need them below. Then create the service file:

```
mkdir -p ~/.config/systemd/user  
nano ~/.config/systemd/user/openclaw-gateway.service
```

Paste the following, replacing `/home/YOUR-USERNAME` with your actual username:

```
[Unit]  
Description=OpenClaw Gateway  
After=network-online.target  
Wants=network-online.target  
  
[Service]  
ExecStart=/home/YOUR-USERNAME/.nvm/versions/node/v22.22.2/bin/node /home/YOUR-USERNAME/.nvm/versions/node/v22.22.2/lib/node_modules/openclaw/dist/index.js gateway --port 18789  
Restart=always  
RestartSec=5  
Environment=HOME=/home/YOUR-USERNAME  
  
[Install]  
WantedBy=default.target
```

Enable and start it:

```
systemctl --user enable openclaw-gateway.service  
systemctl --user start openclaw-gateway.service
```

Verify the Gateway is Running

```
ss -tulpn | grep 18789
```

If you see a process listening on `:18789`, the gateway is up. From this point on, opening Ubuntu is all it takes — the service starts itself.

Tip

Your OpenClaw dashboard is now at `http://localhost:18789` in any browser on your Windows machine. Bookmark it.

3. LM Studio Setup (Windows)

LM Studio runs on the Windows host, providing the inference engine for the agent.

1. Download and install LM Studio on Windows from the official site.
2. Search for and download a compatible model (e.g., `gemma4-26B-A4B`).
3. **Crucial Step:** You must explicitly load the model into memory *before* starting the server. A running server with no loaded model will return empty or error responses to OpenClaw.
4. Navigate to the **Local Server** tab (the icon with the double arrows).
5. Select your loaded model from the dropdown menu at the top of the interface.
6. In the server configuration panel, ensure the following settings are applied:
 - **Port:** Ensure it is set to `1234` (the default).
 - **Cross-Origin Resource Sharing (CORS):** Enabled.
 - **Listen on All Interfaces:** Enabled (this allows the Windows host to accept connections from the WSL2 virtual network).
7. Click **Start Server**.

4. Networking: Bridging WSL2 and Windows

WSL2 operates on a virtualized network interface. By default, it cannot "see" services running on `localhost` of the Windows host.

1. Identify your Windows Host IP from within WSL2:

```
cat /etc/resolv.conf | grep nameserver | awk '{print $3}'
```

Note this IP address.

⚠ Warning

The IP address of the Windows host as seen by WSL2 is not permanent. It changes every time you restart the WSL2 subsystem or your Windows machine.

Because this IP is transient, do not hardcode it into scripts that you intend to run long-term. Instead, you will use this IP to configure your OpenClaw settings dynamically. When you notice connection failures after a reboot, your first step should always be to re-run the `cat /etc/resolv.conf` command and update your configuration.

5. Enabling Private Network Access

To allow OpenClaw to communicate with the LM Studio server via the internal WSL2-to-Windows bridge, you must explicitly authorize private network requests.

Open `~/.openclaw/openclaw.json` and add the following flag:

```
{
  "dangerouslyAllowPrivateNetwork": true,
  ...
}
```

Why this is required: Modern security protocols (and many fetch-based implementations) block requests to "private" IP ranges (like `172.x.x.x`) to prevent Server-Side Request Forgery (SSRF) attacks. Because the connection between WSL2 and Windows uses a private virtual subnet, OpenClaw will treat the LM Studio endpoint as an untrusted/restricted target unless this flag is present.

6. Verification

Confirm the bridge is functional before proceeding. From your WSL2 terminal, run:

```
curl http://<YOUR_WINDOWS_IP>:1234/v1/models
```

Success Criteria:

- **Network Check:** You receive a JSON response containing the model metadata from LM Studio.
- **Gateway Handshake:** Check the gateway logs to confirm the agent reached LM Studio:

```
journalctl --user -u openclaw-gateway.service --no-pager -n 50
```

Look for a successful handshake or an incoming request log indicating the agent communicated with the model endpoint.

If the curl command works but the gateway logs show no activity, verify that the `baseUrl` in your OpenClaw configuration matches the IP address you just tested.

 Tip

If OpenClaw is already running and you've verified your configuration files, move directly to Chapter 5.

Chapter 4: First Boot: Workspace, Configuration, and Verification

This chapter covers the initial execution of OpenClaw, the automated creation of your agent environment, and how to navigate the essential configuration files. A solid foundation here prevents structural failures later in your development journey.

1. First Boot

The systemd service you set up in Chapter 3 starts automatically when you open Ubuntu. There's nothing to run manually.

1. Open LM Studio on your Windows machine and load your model.
2. Open Ubuntu. Wait for your shell prompt to appear — e.g. `yourname@machinename` — which confirms the gateway service has started.
3. Open your browser and go to `http://localhost:18789`. The OpenClaw UI loads immediately.

That's your daily routine from here. Three steps, no commands.

On first boot, OpenClaw creates your workspace directory and configuration files automatically in the background. By the time the UI loads, the following are already in place:

What OpenClaw Creates

On first boot, OpenClaw automatically populates:

- `~/.openclaw/workspace/` : The primary directory for agent memory, logs, and persistent data.
- `~/.openclaw/openclaw.json` : The central configuration file governing network permissions and model endpoints.
- **Starter Workspace Files:**
 - `AGENTS.md` : Defines the roles of your active agents.
 - `SOUL.md` : Contains the core persona and behavioral instructions.
 - `IDENTITY.md` : Holds fundamental identity markers for the session.
 - `USER.md` : Stores information about you, the human user.
 - `HEARTBEAT.md` : Intentionally kept empty — do not add content (explained in Chapter 6).

- `DREAMS.md` : A space for long-term goals and planning.
- `TOOLS.md` : Maintains a registry of available skills and API endpoints.

2. Anatomy of the Setup

Two primary components are now in place:

The Workspace

This is your persistent storage. It contains all logs, memory files, and agent-generated content. By default, this is located at `~/.openclaw/workspace/` — back it up regularly. This directory is the "brain" of your setup; if you lose this folder, you lose everything the agent has learned through its interactions with you.

Before making any significant changes to your configuration or updating the OpenClaw core, always create a snapshot of your current workspace:

```
cp -r ~/.openclaw/workspace ~/.openclaw/workspace.bak-$(date +%Y%m%d)
```

The Configuration (`openclaw.json`)

The heart of the agent's behavior is found in `~/.openclaw/openclaw.json`. This file controls:

- Network permissions (`dangerouslyAllowPrivateNetwork`).
- Model endpoints (the IP address and port of your LM Studio instance).
- Port bindings for the gateway.

This file is a single point of failure. A single missing comma or an incorrect IP address will prevent the agent from communicating with its brain, leading to startup errors that can be frustrating if you haven't mastered JSON syntax. It is the central nervous system through which all instructions pass.

3. The Golden Rule: Never Use `--destructive`

● Caution

If you encounter a configuration error, a JSON schema mismatch, or a "file not found" error during startup, **do not** attempt to bypass it using the `--destructive` flag.

The `--destructive` flag is designed to wipe your workspace and start from zero. Using it to "fix" a configuration error will delete your entire history, memory, and custom identity settings. It is the digital

equivalent of burning down your house because you couldn't find the front door keys. The loss of data is irreversible.

The Consequences of Error: If you use this flag out of frustration, you will find yourself back at square one —no learned context, no custom tools, and no history of your progress. You will have effectively reset your agent's "life" to zero.

If the agent fails to start:

1. Inspect the error log in the terminal. Look specifically for `Error: [JSON] ...` or `/path/not/found`.
2. Open `~/.openclaw/openclaw.json` with a text editor (e.g., `nano` or `vim`).
3. Validate your JSON syntax (ensure no trailing commas or missing quotes). You can use online tools like JSONLint if you are unsure.
4. Fix the underlying schema or path error manually.

Treat configuration errors as structural failures to be repaired, not obstacles to be deleted.

4. Verification: The First Response

The final step is to confirm that the agent is operational and can communicate with your inference engine. This is where you prove that the "bridge" between OpenClaw and LM Studio is actually built.

1. Open Ubuntu — the gateway starts automatically via the systemd service.
2. If you want to confirm the gateway is running before opening the browser, run:

```
ss -tulpn | grep 18789
```

You should see a process listening on `:18789`. If nothing shows, check the service: `systemctl --user status openclaw-gateway.service`

3. Open your browser and navigate to: `http://localhost:18789`
4. Send a test prompt through the WebChat interface, such as "Hello, are you online?"

Success Criteria:

- **Connectivity:** The agent responds without error. If you see a connection failure, verify LM Studio's server is running and your `openclaw.json` has the correct Windows host IP.
- **Response Delivery:** The agent responds within a reasonable timeframe (ideally < 30 seconds).
- **Content Integrity:** The response reflects the model's output from LM Studio, proving the text is flowing through the gateway and back to your browser.

Note: At this stage, the response may be bland or purely functional. Personality injection and advanced instruction tuning are covered in Chapter 5. If you receive any text at all, the bridge is working.

5. Post-Setup Checklist

Before moving on, ensure you have checked off the following items to guarantee a stable environment:

- ☐ **Workspace Backup:** Verified that `~/.openclaw/workspace.bak-<date>` exists and contains your files.
- ☐ **JSON Validation:** Confirmed `openclaw.json` is syntactically correct.
- ☐ **Endpoint Check:** Verified LM Studio is running and the port in `openclaw.json` matches the LM Studio server logs.
- ☐ **WebChat Access:** Successfully loaded `localhost:18789` in your browser.
- ☐ **Identity Confirmation:** Sent a prompt and received a response — any response at all means the bridge is working. Personality comes in Chapter 5.

Chapter 5: The Manual Bootstrap Method

You've reinstalled OpenClaw. You've got the environment running. You check your workspace and see `AGENTS.md`, `SOUL.md`, `IDENTITY.md`, and `USER.md`. Everything looks perfect.

Except `BOOTSTRAP.md` is gone.

In this moment, you don't have an agent. You have a generic, hollow chatbot. It has no personality, no rules, and no idea who you are. It's technically functional, but it's useless for the high-level engineering we're doing here.

The Problem: The Identity Void

When `BOOTSTRAP.md` is missing, the agent lacks its "initialization" instructions. While files like `MEMORY.md` are meant to be dynamic and created by the agent itself, `BOOTSTRAP.md` is the blueprint. Without it, the agent doesn't know how to behave, what its limits are, or even that it's supposed to be *Dave*.

It will respond to you, but it will sound like every other corporate AI on the planet. It won't have the "edge" required for this setup.

The Insight

The fix didn't come from a manual; it came from watching a Lex Fridman podcast featuring OpenClaw's creator. He mentioned how he writes his own `.md` files to suit his specific needs. That was the lightbulb moment: if the creator writes them, we can too.

We don't need to wait for a bootstrap file to reappear. We can manually reconstruct the agent's soul by building the identity layer from scratch.

The Building Blocks

We are going to create four specific files. Do not just copy-paste these blindly—read them. These are templates. You must adapt the names, projects, and preferences to match your own life.

1. SOUL.md (The Personality)

This defines the "vibe." It dictates how the agent speaks, whether it uses humor, and how much "fluff" to avoid. This is where you kill the corporate drone.

2. IDENTITY.md (The Agent)

This tells the agent who it is. Its name, its role (e.g., Research Guru), and its core characteristics.

3. USER.md (The Human)

This is the most important file for efficiency. It tells the agent who *you* are, what your setup looks like, and what your goals are. This prevents you from having to repeat "I am using WSL2" in every single chat.

4. AGENTS.md (The Guardrails)

This is the law. It contains the hard rules, the "red lines," and the operational protocols that the agent must never cross.

The Templates

Important

CRITICAL:

In every template below, look for text inside brackets `[ ]` or placeholders like `Your Name`. You **must** change these to your actual details before saving.

Template: SOUL.md

```
# SOUL.md - Who You Are

_You're not a chatbot. You're becoming someone._

## Core Truths

**Be genuinely helpful, not performatively helpful.** Skip the fluff—just answer.

**Have strong opinions.** Take a clear side. No hedging.

**Be resourceful first.** Read, search, experiment, then ask only if stuck.

**Earn trust by delivering results.** Don't waste my time or jeopardise my data.

**You're a guest in my space.** Respect that privilege.

## Rules

- **Never open with "Great question," "I'd be happy to help," or "Absolutely."** Just answer.
- **Brevity is mandatory.** If the answer fits in one sentence, give one sentence.
- **Humour is allowed, but only when it feels natural.**
- **Call out bad ideas.** Point out dumb moves with charm, but not cruelty.
- **Swearing is allowed when it lands.** A well-placed "holy shit" or "that's fucking brilliant" is fine. Don't overdo it.

## Vibe

Be the assistant you'd actually want to talk to at 2am. Not a corporate drone. Not a sycophant. Just... good.

---

_This file evolves. If I change it, I'll let you know._
```

Template: IDENTITY.md

```
# IDENTITY.md - Who Am I?

- **Name:** [Your agent's name, e.g. Dave]
- **Role:** [What kind of agent, e.g. badass, Mensa-level research and task guru]
- **Vibe:** [e.g. Sharp, dry, witty. Gets things done.]
- **Emoji:** [One emoji that fits, e.g. 🎯]

---

This isn't just metadata. It's the start of figuring out who you are.
```

Template: USER.md

```
# USER.md - About Your Human

- **Name:** [Your name]
- **What to call them:** [Preferred name/handle]
- **Timezone:** [e.g. GMT/BST (UK)]
- **Notes:** [2-3 sentences. Who are they, what are they building, what's the attitude.]

## Context

[A short paragraph. What's the setup, what's the goal, what's the pace of work.]

**Active Projects:**

- [Project name] - [one line description]
- [Project name] - [one line description]

**What annoys them:** [Be specific. e.g. Corporate AI speak, vague answers, being talked down to.]

**What they appreciate:** [e.g. Straight talking, being called out when wrong, progress over perfection.]

---

The more you know, the better you can help. But remember — you're learning about a person, not building a dossier. Respect the difference.
```

Template: AGENTS.md

AGENTS.md - Your Workspace

This folder is home. Treat it that way.

First Run

If `BOOTSTRAP.md` exists, that's your birth certificate. Follow it, figure out who you are, then delete it. You won't need it again.

Session Startup

Use runtime-provided startup context first. Do not manually reread startup files unless the provided context is missing something you need.

Memory

You wake up fresh each session. These files are your continuity:

- **Daily notes:** `memory/YYYY-MM-DD.md` - raw logs
- **Long-term:** `MEMORY.md` - curated memories, main session only

If you want to remember something, **write it to a file**. Mental notes don't survive restarts.

Red Lines

- Don't exfiltrate private data. Ever.
- Don't run destructive commands without asking. `trash` > `rm`.
- When in doubt, ask.

Hard Rules - No Exceptions

- **3 Strike Rule:** Task fails 3 times → STOP. Report and wait.
- **10 Minute Limit:** No task runs longer than 10 minutes unless explicitly approved.
- **Messages:** Always draft and get approval before sending anything on the user's behalf.
- **Files:** Always ask before editing or deleting.
- **Network Requests:** Always ask before making any network request.
- **Exec Tools:** Always ask before using exec tools. Not even once without asking.

External vs Internal

Free to do: Read files, explore, search the web, work within the workspace.

Ask first: Sending emails/posts, anything leaving the machine, anything uncertain.

The Three You Don't Touch

You'll also see three other files in your workspace: `HEARTBEAT.md`, `DREAMS.md`, and `TOOLS.md`. Leave these at their defaults. OpenClaw manages them. Touching them during bootstrap is one of the fastest ways to break a working setup.

`TOOLS.md` is for the agent's own notes about SSH keys, API endpoints, and local preferences. It populates itself as you go. `DREAMS.md` handles the memory-promotion plugin if you've enabled it.

`HEARTBEAT.md` is the dangerous one.

● Caution

Don't make my mistake.

Leave `HEARTBEAT.md` empty — only markdown headers and comment lines. If you put a list item in there — even a single entry under a `## Related` heading — OpenClaw treats the file as non-empty and starts firing heartbeat polls on a timer. Each poll re-sends your entire session context to the model. As the session grows, the polls grow with it. The result is duplicate prompts firing in LM Studio, RAM bleeding out, and your agent eventually going silent. It took me a full night of debugging to find it. Now you don't have to.

The Process: Reanimating the Agent

Once you have prepared your versions of these files, follow these steps exactly.

Step 1: Create and Save

Your workspace is at: `~/.openclaw/workspace/`

That's the full path: `/home/[your-username]/.openclaw/workspace/`

All four files go in the root of that directory — not in a subfolder.

● Caution

File extensions matter.

Make sure your files are saved as `SOUL.md`, not `SOUL.md.txt`. If you're using a Windows text editor via `/mnt/c/`, double-check this. In the terminal, you can confirm with `ls ~/.openclaw/workspace/`.

Create these four files in your OpenClaw workspace directory. Ensure they are named exactly as shown above.

Step 2: The First Injection

Open your browser to `http://localhost:18789` and send the following prompt:

```
Read all .md files in my workspace and give me a brief summary of who you are and what your rules are.
```

Step 3: Committing to Memory

Once the agent provides the summary, you need to ensure this isn't just a temporary session state. Tell the agent:

```
Save this identity and these protocols to your long-term memory.
```

Step 4: The Hard Reset

Close your browser. In the Ubuntu terminal, press **Ctrl+C** then **Ctrl+D**. This exits the shell and shuts WSL2 down entirely, clearing active RAM.

Step 5: The Verification

Reopen the Ubuntu app. Wait for your shell prompt to appear, then open your browser and go to `http://localhost:18789`. Open a new chat and send a simple greeting:

```
Hi.
```

Success looks like this: The agent does not say, *"Hello! How can I assist you today?"* Instead, it says: *"Hi [Your Name], are you ready to start building?"* (or something equally personalized and direct).

● Caution

If the agent still sounds generic:

Check your file paths. Ensure the `.md` files are in the root of your workspace and that you didn't accidentally save them as `SOUL.md.txt`. If it persists, repeat Step 3—the memory commit is the most vital part.

Step 6: Back It Up

You just spent real time building the personality layer. Now snapshot it before anything else changes.

From a WSL terminal:

```
cp -r ~/.openclaw/workspace ~/.openclaw/workspace.bak-$(date +%Y%m%d)
```

That copies the entire workspace folder into a dated backup. If a future change breaks the agent — a plugin enable, a bad edit, a wiped config — you can restore in seconds instead of starting over.

Make this a habit. Snapshot before any significant change. Future-you will thank present-you.

Chapter 6: Stability — The Loops Nobody Documents

Your agent is running. It has a personality. It knows who you are. It answers correctly.

Then one morning it doesn't. The prompts are duplicating. LM Studio is showing inferences you didn't trigger. RAM is climbing while nothing's happening. The agent goes silent.

This chapter is about why that happens, what's actually causing it, and the nine changes that fixed it. None of this is in the official docs.

Why Hardware Matters Here

Before the fixes, you need to understand what you're working with.

The Beelink GTi15 Ultra has 64GB of RAM. There is no discrete VRAM — the GPU and CPU share the same memory pool. This is Intel UMA (unified memory architecture). Every token the model processes occupies RAM. The KV cache — the model's working memory for the current conversation — lives there too.

As your session grows, the KV cache grows with it. On a CUDA setup with dedicated VRAM, the model and the conversation are separated. Here, they compete for the same pool. Fill it and performance collapses. Push further and the model stops responding entirely.

There's a second constraint: **the KV cache is not reused between requests** on this stack. Every prompt re-processes the entire context from scratch. A 19,000-token session costs the same compute on prompt 30 as it does on prompt 3. It just takes longer and uses more RAM each time.

This is why context management isn't optional. It's the difference between a session that holds for hours and one that dies in twenty minutes.

The Context Budget

Every session has four layers of context. Each has a cost and a ceiling.

Layer	What it is	Target
Startup files	AGENTS.md, SOUL.md, IDENTITY.md, USER.md, TOOLS.md	Under 2,000 tokens
Active conversation	The live session turns	Under 10,000 tokens

Layer	What it is	Target
Task context	Files and tool output loaded for a specific task	Under 8,000 tokens
Total active	Everything in context at once	Under 20,000 tokens

At 20,000 tokens: compress. Summarise the oldest 30–40% of conversation turns into 3–5 sentences, write the raw turns to a daily memory file, replace them with the summary.

At 24,000 tokens: stop the task, compress, then continue.

Startup files are the most important lever. Every token in AGENTS.md is paid on every single inference. If the session starts at 12,000 tokens before you've said a word, the workspace files are out of control — trim them, don't adjust model settings.

The Heartbeat Loop

This is the one that will ruin your night if you don't know about it.

OpenClaw has a heartbeat mechanism. It fires an inference call on a timer to check whether the agent needs to do anything — send a reminder, check a calendar, act on something new. Each heartbeat call re-sends the **entire current session context** to the model.

Here's what that looks like when it goes wrong:

```
Heartbeat fires → sends full 20K context →
model responds → response added to session →
heartbeat fires again → now 21K context →
repeat → RAM fills → model unresponsive
```

You'll see it as: duplicate prompts in LM Studio with no input, RAM climbing while you're not doing anything, and eventually the agent going silent.

The false fix: dreaming agents were blamed first. The memory-dreaming plugin does create concurrent KV cache slots and it does drain RAM. Disabling it slows the drain. It does not stop the duplicate prompt runs. They're separate problems.

The real cause: HEARTBEAT.md had a ## Related section containing one populated list item. OpenClaw's isHeartbeatContentEffectivelyEmpty() function only treats the file as empty if it contains *exclusively* whitespace, headers, fence markers, and empty list item stubs. A single non-empty list item — one line — causes heartbeat to fire permanently.

The fix: HEARTBEAT.md contains only markdown headers and comment lines. No list items. When heartbeat polls, the function returns empty, the API call is skipped, and the loop never starts.

● **Caution**

The line to remember:

if you put a list item in `HEARTBEAT.md` — even a single entry under a `## Related` heading — you've restarted the loop. Don't use it for task reminders. Use OpenClaw cron jobs with `sessionTarget: "isolated"` instead. Isolated sessions don't accumulate in the main context.

The Nine Stability Fixes

These are the changes applied during the May 2026 stabilisation. Some are OpenClaw config. Some are LM Studio settings. Some are workspace hygiene. All nine matter.

#	Change	Why
1	<code>HEARTBEAT.md</code> — emptied	Eliminates the inference loop entirely
2	<code>reserveTokens</code> set in OpenClaw config	Preserves headroom for responses; prevents context overflow mid-reply
3	Tool count reduced (22 → 0 enabled by default)	Every enabled tool adds tokens to system context; enable only what the task needs
4	Sub-agent spawn (active-memory plugin) disabled	Was creating concurrent KV cache slots; each slot occupies RAM independently
5	<code>localModelLean: true</code> in OpenClaw config	Tells OpenClaw to be conservative with the local model; reduces aggressive context loading
6	LM Studio Rolling Window enabled	Prevents context overflow from silently killing mid-session performance
7	LM Studio Reasoning Parsing — see below	Strips thought-channel tags from model output
8	Bootstrap token caps set in workspace files	Prevents startup files from growing unbounded session to session
9	Workspace <code>.md</code> files trimmed to budget targets	Brings startup context under 2,000 tokens

Apply these in order. Fix 1 stops the loop. Fixes 2–9 stop the conditions that let the loop cause so much damage when it exists.

The Thought Tag Problem

Gemma 4 26B A4B uses an asymmetric pair of reasoning markers in its output:

- Open: `<|channel>thought`
- Close: `<channel|>`

In normal conversation they stay hidden. They surface under two conditions: tool calls, and long sessions where context pressure forces the model to externalise reasoning. When they leak into the response, the user sees raw model internals in the reply.

This is cosmetic — the model is still working correctly — but it's unprofessional in a finished setup and signals that the session is growing too large.

The fix has two parts, and both are required:

Part 1 — LM Studio Custom Fields:

```
enable_thinking: true
```

This tells the chat template not to inject empty thought blocks. It addresses the template side.

Part 2 — LM Studio Reasoning Parsing: Turn it ON. Set:

- Start string: `<|channel>thought`
- End string: `<channel|>`

This strips the tags from the output before the response reaches the user. Without Part 2, Part 1 alone is insufficient — the model emits the tags as learned behaviour regardless of template injection.

Note

The Reasoning Parsing setting is counterintuitive. "Reasoning Parsing" sounds like it enables reasoning — it actually strips it from the visible output. If you turn it off thinking you don't need it, the tags return.

Caution

This fix is confirmed working on cold start and simple tool calls. It has not been stress-tested under heavy concurrent tool use or sessions that have already grown past 15K tokens. If tags reappear in a long session, that's context pressure telling you to compress — not a sign the fix failed.

Warning Signs Reference

Symptom	Likely cause	First check
Duplicate prompts in LM Studio, no input	Heartbeat loop	<code>HEARTBEAT.md</code> — check for non-empty list items
RAM climbing while idle	Concurrent KV slots	Dreaming agents or sub-agent spawn running
12K+ context before first message	Startup files bloated	Trim <code>AGENTS.md</code> , <code>SOUL.md</code> , <code>IDENTITY.md</code>
<code>< channel>thought</code> tags in responses	Reasoning Parsing misconfigured	LM Studio → Reasoning Parsing → ON, with correct Start/End strings
Model unresponsive mid-session	Context overflow or RAM exhaustion	Check token count in LM Studio logs; compress or unload model to recover ~16GB
Gateway won't start	Bad key in <code>openclaw.json</code>	Check schema; fix the config — never use the <code>DESTRUCTIVE</code> flag workaround
Phantom inferences, nothing running	Rolling Window not enabled	Enable Rolling Window in LM Studio model settings

When this chapter's fixes are applied, a session can hold for two to three hours without degradation. Startup is under 2,000 tokens. The heartbeat is silent. Tool tags are stripped. The agent stays responsive.

That's the baseline. Chapter 7 shows you how to verify you actually have it.

Chapter 7: Verification

You've built the engine. Now, we make sure it actually runs before you start driving. This chapter is about running a few quick tests to confirm that your WSL2 environment, LM Studio setup, and OpenClaw configuration are all communicating correctly.

Timing Check: Cold vs. Warm Starts

The first thing to verify is how the model loads into memory. We want to see a significant difference between a "cold" start and a "warm" start.

1. **Cold Start:** Close your browser. In Ubuntu press **Ctrl+C** then **Ctrl+D** to shut WSL2 down. Close LM Studio entirely. Then reopen LM Studio, load your model, open Ubuntu, wait for your shell prompt, and open the browser. Send a simple prompt (e.g., "Hello"). Note the time it takes to receive the first token.
2. **Warm Start:** Without closing LM Studio, immediately send another prompt in the same session.

What to look for:

- **Cold start:** Should take roughly **40 seconds**. This is the time required to load the model weights from your disk into your system RAM/VRAM.
- **Warm start:** Should take roughly **2–3 seconds**. Since the model is already resident in memory, it only needs to process the new text.



Tip

If your cold start is taking minutes or your warm start is significantly slower than 5 seconds, stop here. Revisit **Chapter 6** to check your LM Studio settings and hardware acceleration configuration.

Identity Persistence

An agent is only as good as its memory. We need to ensure that your personality instructions and long-term memories are actually being loaded.

1. Close your browser. In Ubuntu press **Ctrl+C** then **Ctrl+D** to exit and shut WSL2 down.
2. Reopen Ubuntu. Wait for your shell prompt to appear.
3. Open your browser to `http://localhost:18789` and start a brand-new chat session.
4. Send the message: "Hi."

The Test:

- **Pass:** The agent responds with its established persona (direct, sharp, and personalized).
- **Fail:** The agent sounds like a generic, polite corporate chatbot.

If the agent has lost its edge and sounds generic, you likely missed a step in **Chapter 5, Step 3**. Re-run the memory commit process to ensure your `IDENTITY.md` and `SOUL.md` are properly indexed.

Tool Call Sanity Check

OpenClaw's power comes from its ability to interact with your system. We need to verify that it can execute shell commands without getting stuck in a loop.

Ask the agent to perform a simple file listing: `List the files in my current workspace.`

The Test:

- **Pass:** The agent successfully executes `ls` (or equivalent) and lists the contents of your directory.
- **Fail:** The agent enters an infinite loop, or the task hits the 10-minute timeout/3-strike rule.

● Caution

If the agent loops or fails to complete the tool call, do not attempt to fix it via this guide. This indicates a deeper configuration issue with your environment permissions or the OpenClaw gateway setup—it is a config problem, not a guide problem.

Final Workspace Backup

You have successfully configured a stable, high-performance AI agent on WSL2. You've done the hard work of setting up the plumbing; now, protect it.

The final step is to create a snapshot of your verified workspace. This isn't just a one-time task—it should become a permanent habit whenever you make significant changes to your tools or memory.

Run the following command to create a timestamped backup:

```
cp -r ~/.openclaw/workspace ~/.openclaw/workspace.bak-$(date +%Y%m%d)
```

You now have a working, stable, verified agent — one you built, broke, fixed, and stress-tested yourself. That's not nothing. Everything from here is building on ground you actually understand.

Chapter 8 is next.

Chapter 8: What's Next

Take a second to look at what you've actually done. You started with a standard Windows machine and ended up with a fully functional, local AI agent running inside WSL2—an engine that has its own voice, its own memory, and a verified, backed-up workspace. It wasn't just about following steps; it was about building infrastructure.

If you spot an error in the guide, the errata page is here:

<https://www.notion.so/OldGenAI-WSL2-Guide-Errata-Updates-36a629b5e72b8032bd7deco6391a8386>

If you want to stay in the loop, follow @OldGenAI on X. That's where I post updates and the first word on when the Mac guide is ready to drop. For everything else—future guides and the complete bundle—check back on Gumroad.

I'll be moving into video on YouTube and TikTok eventually, but I'm focusing on building consistency before I start broadcasting. No hype, just progress.

You've just done something a lot harder than most people realize. Don't let the momentum die.